

問題 1.1 2 進数への変換と IEEE754 規格浮動小数点表現

$(123.40625)_{10}$ を 2 進数へ変換しなさい。さらに、IEEE754 規格の浮動小数点表現（単精度 32 ビット）で書きなさい。

回答

$$\begin{aligned}(123.40625)_{10} &= 64 + 32 + 16 + 8 + 2 + 1 + 0.25 + 0.125 + 0.03125 \\&= 2^6 + 2^5 + 2^4 + 2^3 + 2^1 + 2^0 + 2^{-2} + 2^{-3} + 2^{-5} \\&= (1111011.01101)_2 \\&= (1.11101101101)_2 \times 2^6\end{aligned}$$

符号が正であるため、符号ビットは 0 になる。単精度のため、指数に 127 を足し、指数ビットは $6 + 127 = (133)_{10} = (10000101)_2$ である。で正規化した 1 を除き、仮数部は $11101101101\bar{0}$ である。

$$(123.40625)_{10} \xrightarrow{\text{IEEE754 single precision}} 0\ 10000101\ 111011011010000000000000$$

問題 1.2 丸め誤差

浮動小数点数の有効桁は、単精度と倍精度ではおよそ何桁か計算しなさい。また、実際に 0.1 を単精度および倍精度変数に代入し、何桁まで有効であるか確認しなさい

回答

単精度では仮数部が 23 ビットのため、2 進数で有効桁数が 24 桁（1 の位も含め）であり、10 進数で有効桁数が $\log_{10} 2^{24} = 24 \log_{10} 2 \approx 7$ 桁である。

倍精度では仮数部が 52 ビットのため、2 進数で有効桁数が 53 桁であり、10 進数で有効桁数が $\log_{10} 2^{53} = 52 \log_{10} 2 \approx 15$ 桁である。

ソースコード

```
//1.2.c
#include <stdio.h>
int main(int argc, char **argv){
    float x32 = 0.1; double x64 = 0.1;
    printf("%.16e %.16e\n", x32, x64);
    return 0;
}
```

実行結果

環境 : wsl

```
/mnt/c/Users/User/Desktop/numerical_analysis$ ./1.2
1.00000000149011612e-01 1.000000000000000001e-01
```

考察

プログラムでは、単精度変数 x32、倍精度変数 x64 それぞれに 0.1 を代入して浮動小数点以下 16 桁の指数表記で出力する。実行結果から、単精度変数は 7 桁まで、倍精度変数は 15 桁まで有効であると分かる。

問題 1.3 オーバーフロー・アンダーフロー

2^n を整数型で計算し、整数型の表現できる最大数を確認しなさい。この結果から、使用したシステムでは整数型は何ビットであるか検討しなさい。

回答

整数型がオーバーフローすると値が増えず、負の値に逆転する。以下のソースコードで整数型 `int` のビット数を確認する。変数 $x \leftarrow 2^n$ を繰り返して計算し、 x がオーバーフローするとループから抜き出す。

ソースコード

```
//1.3.c
#include <stdio.h>
int main(int argc, char** argv){
    int x = 1;
    long int xl = 1;
    int n = 0;
    do {
        x *= 2;
        xl *= 2; n++;
        printf("%d\t%12d\t%12ld\n",n,x,xl);
    } while(x != 0);
    return 0;
}
```

このプログラムでは、整数型変数 `x` と長整数型変数 `xl` で 2^n を計算する。長整数型変数のビット数がより大きく、より大きい範囲の値に対応するため、`x` がオーバーフローするとき、`xl` を真値として比較することができる。

実行結果

環境 : wsl

User/Desktop/numerical_analysis\$./1.3			17	131072	131072
1	2	2	18	262144	262144
2	4	4	19	524288	524288
3	8	8	20	1048576	1048576
4	16	16	21	2097152	2097152
5	32	32	22	4194304	4194304
6	64	64	23	8388608	8388608
7	128	128	24	16777216	16777216
8	256	256	25	33554432	33554432
9	512	512	26	67108864	67108864
10	1024	1024	27	134217728	134217728
11	2048	2048	28	268435456	268435456
12	4096	4096	29	536870912	536870912
13	8192	8192	30	1073741824	1073741824
14	16384	16384	31	-2147483648	2147483648
15	32768	32768	32	0	4294967296
16	65536	65536			

考察

実行結果から、 2^{31} で負の値に逆転し、 2^{32} でオーバーフローしたことが分かった。 2^n は初期値 $2^0 = 1$ を左に n 回ビットをシフトすることと同等であることから、 2^{31} を計算した結果、符号ビットが 1 になり、負に逆転した。すなわち、この実行環境で整数型は 32 ビット長いことが分かる。

問題 1.4 打ち切り誤差

ネイピア数 e は無限級数 $e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$ で表されるが，打ち切り誤差を 10^{-10} 以下にするためには何項目までを計算すればよいかを，実際に計算して確認しなさい。

ソースコード

```
//1.4
#include <stdio.h>
#include <math.h>
long int fact(long int x){
    if(x == 0) return 1;
    else return x * fact(x - 1);
}
int main(int argc, char** argv){
    double e = 0;
    long int i = 0;
    const double max_err = 0.0000000001;
    double err;
    while(1) {
        e = e + (1 / (double) fact(i));
        err = M_E - e;
        printf("%ld %e\n",i,err);
        i++;
        if(fabs(err) <= max_err) break;
    }
}
```

このプログラムでは、math.h で定義されている定数 M_E を真値として比較する。変数 e と定数 M_E の誤差 err が 10^{-10} 以下になるまで、変数 e の近似値を一つずつ項目増やして計算する。

実行結果

実行環境 : wsl

```
/mnt/c/Users/User/Desktop/numerical_analysis$ ./1.4
```

```
0 1.718282e+00
```

```
1 7.182818e-01
```

```
2 2.182818e-01
```

```
3 5.161516e-02
```

```
4 9.948495e-03
```

```
5 1.615162e-03
```

```
6 2.262729e-04
```

```
7 2.786021e-05
```

```
8 3.058618e-06
```

```
9 3.028859e-07
```

```
10 2.731266e-08
```

```
11 2.260552e-09
```

```
12 1.728764e-10
```

```
13 1.228573e-11
```

考察

13 項目まで計算すると、変数 e と定数 M_E の誤差が 10^{-10} 以下になった。すなわち、単精度変数で 10 桁まで有効である近似値を得るには、13 項目以上計算する必要がある。

課題 1.5 誤差の蓄積・伝播

0.1 を 10000 回加算したときに生じる誤差の蓄積・伝播を、単精度および倍精度変数で計算し、図 1 のように可視化しなさい。

ソースコード

```
//1.5.c
#include <stdio.h>

long double lfabs(long double x){
    if(x < 0) return -x; else return x;
}

int main(int argc, char** argv){
    float x32; double x64;
    long double x128, err32, err64;
    for(int i = 1; i <= 10000; i++){
        x128 += 0.1; x64 += 0.1; x32 += 0.1;
        err32 = x128 - (long double) x32;
        err64 = x128 - (long double) x64;
        if(i % 10 == 0)
            printf("%d\t%.30Le\t%.30Le\n",i,lfabs(err32),lfabs(err64));
    }
    return 0;
}
```

誤差が単精度と倍精度の有効桁を超えるため、128 ビット長い long double 変数を真値として各回で生じた誤差を計算し、10 回ごとにその絶対誤差を 30 桁の指数表記で出力する。対数グラフで可視化するため、絶対誤差を取る。出力を errors.csv にリダイレクトし、Python の matplotlib モジュールで可視化する。

実行結果

実行環境 : wsl

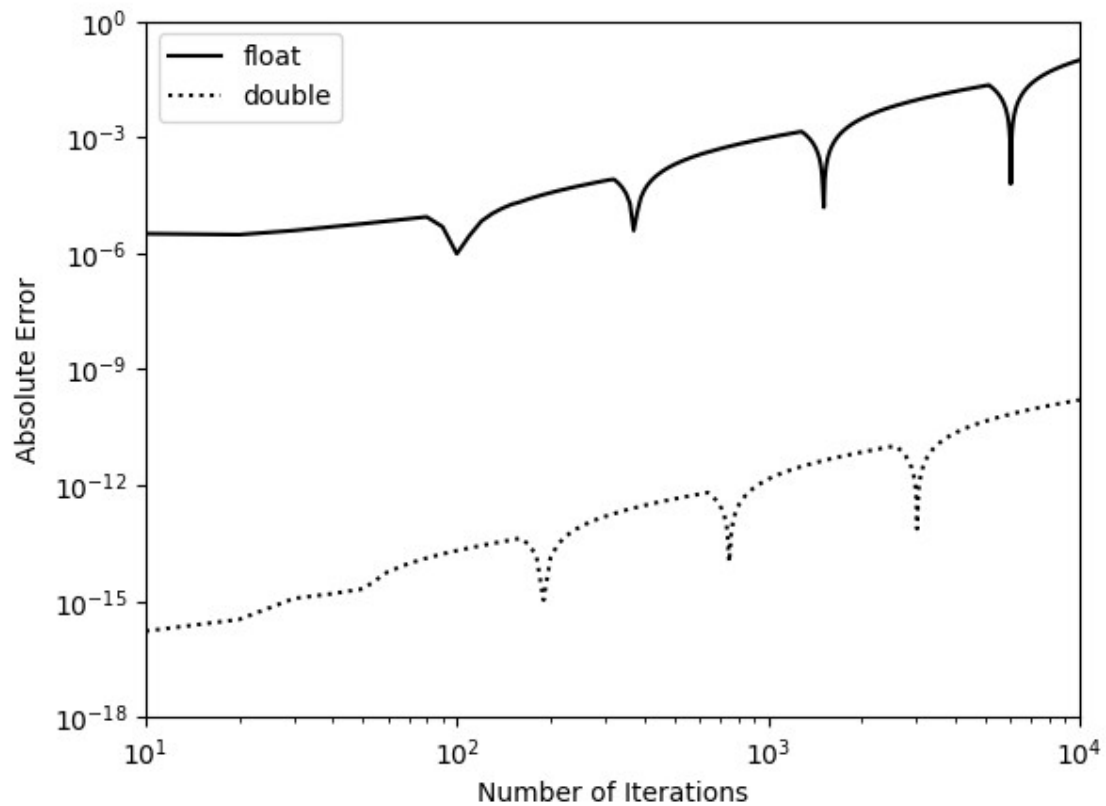


図 1 誤差の蓄積・伝播

考察

グラフから倍精度の絶対誤差が単精度より低いに見える。ループ回数が大きくなると両方の誤差が拡大し、誤差が蓄積していることが分かった。

問題 1.2 で単精度の有効桁数が 7 つつまり誤差が 10^{-7} より小さく、倍精度の誤差が 10^{-15} より小さいことが分かった。最初の 10 回目で、単精度の誤差が 10^{-5} と 10^{-6} の間から、倍精度の誤差が 10^{-15} と 10^{-16} の間から始まるように見えるは、10000 回計算したらそれぞれの誤差がと 10^{-3} と 10^{-12} を超えた。浮動小数点を演算し続けて誤差が蓄積・伝播し、精度が低くなると解釈できる。

課題 1.6 (自由課題)

n を 1 から 10000 まで $1/n$ を単精度と倍精度で計算し、生じた誤差を可視化する

回答

ソースコード

```
//1.6.c
#include <stdio.h>

long double lfabs(long double x){
    if(x < 0) return -x;
    else return x;
}

int main(int argc, char** argv){
    float x32; double x64; long double x128;
    long double err32, err64;
    for(int n = 1; n <= 10000; n++){
        x128 = 1/ (long double)n;
        x64 = 1/ (double)n;
        x32 = 1/ (float)n;
        err32 = x128 - (long double) x32;
        err64 = x128 - (long double) x64;
        if(n % 10 == 0)
            printf("%d\t%.30Le\t%.30Le\n",n,lfabs(err32),lfabs(err64));
    }
    return 0;
}
```

問題 1.5 と同様に、多倍長浮動小数点数 long double で誤差を計算する。x32, x64, x128 を演算するとき違う型キャストを明確に書いてある。型キャストがないと $1/n$ が 0 になり、型変換されて変数に代入されるからである。

実行結果

実行環境 : wsl

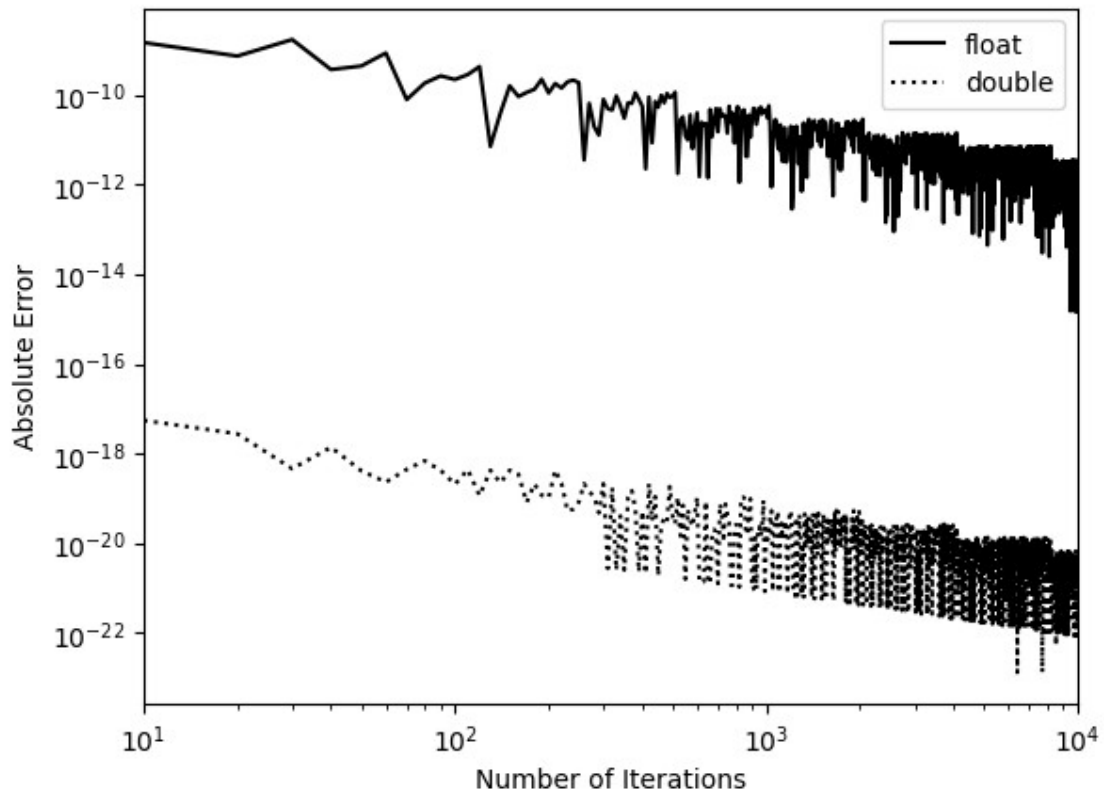


図 2 誤差の蓄積・伝播

考察

問題 1.5 と同様に、グラフから倍精度の絶対誤差が単精度より低いに見える。ただし、ループ回数が増えれば増えるほど誤差が小さくなる。誤差が減少する理由は、 n が大きくなれば、 $1/n$ が 0 に収束するからである。

検討事項として $1/n$ が単精度と倍精度で表現できない n の上限を考える。 $1/n$ が 0 に収束するため、IEEE754 の特殊表現にならないように、指数部の最下位ビットを 1、それ以外を 0 にする。仮数部は最も小さい値にして、全ビットを 0 とする。符号ビットを 0 にする。

$1/n$ が IEEE754 で表現できる n の上限を N 、IEEE754 で表せる値の下限を val_{min} とする。これにより、 $0 < \frac{1}{N} < val_{min}$

単精度の場合、

$$\begin{aligned}val_{min} &= (1.0)_2 \times 2^{1-127} \\&= (1.0)_2 \times 2^{-126} \\&= 1.0 \times 10^{-126 \log_{10} 2} \\&\approx 10^{-37.9298}\end{aligned}$$

$$N^{-1} < 10^{-37.93}$$

$$N > 10^{37.93}$$

倍精度の場合、

$$\begin{aligned}val_{min} &= (1.0)_2 \times 2^{1-1023} \\&= (1.0)_2 \times 2^{-1022} \\&= 1.0 \times 10^{-1022 \log_{10} 2} \\&\approx 10^{-307.6527}\end{aligned}$$

$$N^{-1} < 10^{-37.66}$$

$$N > 10^{307.66}$$